照相机

照相机
例程实验效果简介
代码讲解

例程实验效果简介

> 在后续训练自己的AI视觉模型的时候，建议用K230拍摄待识别的目标，保存图片作为训练集。
>
> 这样可以优化减少一些由于训练的照片与实际识别的环境不同而造成的效果欠佳

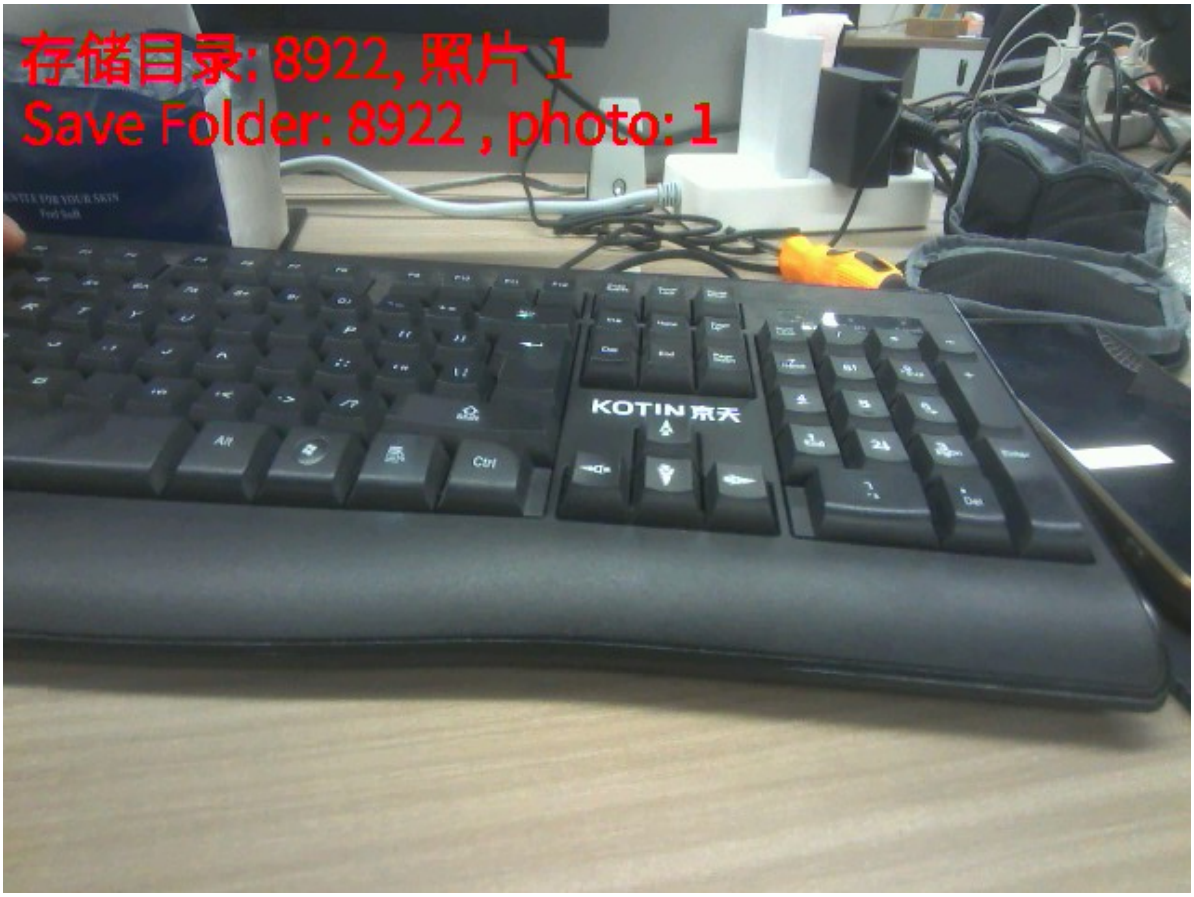
本节例程代码位于：[源码汇总 / 02.Basic / 19.snapshot.py]

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

在系统的文件管理器中，打开CanMV对应的目录。我们进入 CanMV/sdcard这个目录里面

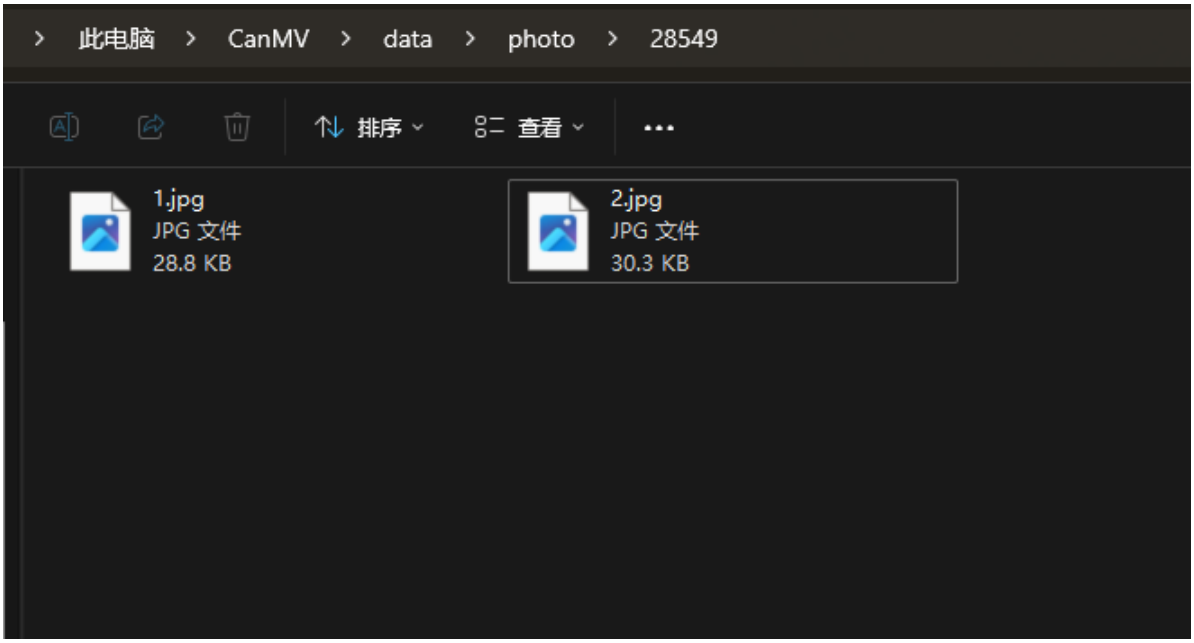
拍照保存的图片路径在 [/sdcard/snapshot/] 目录下

运行程序，摄像头会被唤起，屏幕上显示当前正在拍摄的照片是第几张照片



按下Key键，K230会拍下当前的画面，并保存在【/sdcard/snapshot/（xxx）】目录下，每次运行保存的文件夹 (xxx) 都会显示在屏幕上

以上图的代码为例，本次拍摄的照片保存路径为 [/sdcard/snapshot/28549/]



代码讲解

本节例程代码如下

```
# 导入必要的系统和操作系统模块
# Import necessary system and OS modules
import uos

import time
from media.sensor import *
from media.display import *
from media.media import *
import ybUtils.YbKey as YbKey

# 定义图片保存路径和文件命名相关变量
# Define variables for image saving path and file naming
save_path = "/data/snapshot/"  # 保存基础路径 Base saving path
prefix = time.ticks_us() % 10000  # 使用时间戳作为文件夹名 Use timestamp as folder
name
i = 1  # 照片计数器 Photo counter

# 保存图像原始数据，可使用7yuv预览
# save image raw data, use 7yuv to preview

def ensure_dir(directory):
    """
    递归创建目录
    (Recursively create directory)
    """
    # 如果目录为空字符串或根目录，直接返回
    # (If directory is empty string or root directory, return directly)
    if not directory or directory == '/':
        return

    # 处理路径分隔符，确保使用标准格式
    # (Process path separators to ensure standard format)
    directory = directory.rstrip('/')
```

```
    try:
        # 尝试获取目录状态，如果目录存在就直接返回
        # (Try to get directory status, if directory exists then return
directly)
        os.stat(directory)
        print(f'目录已存在: {directory}')
        # (Directory already exists: {directory})
        return
    except OSError:
        # 目录不存在，需要创建
        # (Directory does not exist, need to create)

        # 分割路径以获取父目录
        # (Split path to get parent directory)
        if '/' in directory:
            parent = directory[:directory.rindex('/')]
            if parent and parent != directory:  # 避免无限递归
                                                # (Avoid infinite recursion)
                ensure_dir(parent)

        try:
            # 创建目录
            # (Create directory)
            os.mkdir(directory)
            print(f'已创建目录: {directory}')
            # (Directory created: {directory})
        except OSError as e:
            # 可能是并发创建导致的冲突，再次检查目录是否存在
            # (Possible conflict due to concurrent creation, check again if
directory exists)
            try:
                os.stat(directory)
                print(f'目录已被其他进程创建: {directory}')
                # (Directory has been created by another process: {directory})
            except:
                # 如果仍然不存在，则确实出错了
                # (If it still doesn't exist, there is definitely an error)
                print(f'创建目录时出错: {e}')
                # (Error creating directory: {e})
    except Exception as e:
        # 捕获其他可能的异常
        # (Catch other possible exceptions)
        print(f'处理目录时出错: {e}')
        # (Error processing directory: {e})

if __name__ == "__main__":
    try:
        # 使用默认配置构造传感器对象
        # Construct a Sensor object with default configuration
        key = YbKey.YbKey()

        sensor = Sensor()
        # 重置传感器
        # Reset sensor
        sensor.reset()

        # 设置通道1的输出格式
```

```
        # Set channel 1 output format
        sensor.set_framesize(width = 640, height = 480, chn = CAM_CHN_ID_1)
        sensor.set_pixformat(Sensor.RGB565, chn = CAM_CHN_ID_1)

        Display.init(Display.ST7701,width=640,height=480,to_ide = True)

        # 初始化媒体管理器
        # Initialize media manager
        MediaManager.init()
        # 启动传感器
        # Start sensor
        sensor.run()

        last_status = False
        while True:
            # 捕获图像
            # Capture image
            img = sensor.snapshot(chn = CAM_CHN_ID_1)
            # 创建显示用的图像缓冲
            # Create image buffer for display
            img2 = image.Image(640, 480, image.RGB565)
            img2.clear()
            img2.copy_from(img)
            # 在图像上绘制信息文本
            # Draw information text on image
            img2.draw_string_advanced(10, 10, 30,"存储目录: " + str(prefix)+", 照片
" + str(i) + " ", color=(255, 0, 0))
            img2.draw_string_advanced(10, 45, 30,"Save Folder: " + str(prefix)+"
, photo: " + str(i) + " ", color=(255, 0, 0))
            Display.show_image(img2, 0, 0, Display.LAYER_OSD2)

            key = YbKey.YbKey()
            # 按键检测和图片保存逻辑
            # Button detection and image saving logic
            if key.is_pressed() == 1:
                if last_status == False:
                    last_status = True
                    ##########################
                    ensure_dir(save_path + str(prefix) + "/")

                    path = save_path + str(prefix) + "/" + str(i) + ".jpg"
                    i = i+1
                    print(path)
                    img.save(path)
                    print("已保存至: " + path)
                    time.sleep_ms(1)
                    ##########################
            else:
                last_status = False
    except KeyboardInterrupt as e:
        print(f"用户停止程序 User stopped the program")
    except BaseException as e:
        print(f"发生异常 Exception occurred: '{e}'")
    finally:
        # 清理资源和退出程序
        # Clean up resources and exit program

        # 停止传感器
```

```
        # Stop sensor
        if isinstance(sensor, Sensor):
            sensor.stop()
        # 反初始化显示
        # Deinitialize display
        Display.deinit()

        os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
        time.sleep_ms(100)

        # 释放媒体缓冲
        # Release media buffer
        MediaManager.deinit()
```

具体工作流程：

1. 初始化设置：
    - 设定照片保存路径为"/data/snapshot/"
    - 用当前时间戳创建独特的文件夹名
    - 设置照片计数器
2. 相机设置：
    - 初始化摄像头传感器，设置分辨率为800x480
    - 设置RGB565颜色格式
    - 配置HDMI显示输出
    - 初始化媒体管理器
3. 主循环功能：
    - 持续捕获摄像头画面
    - 在预览画面上叠加显示当前拍照信息（保存路径和照片序号）
    - 监测按键输入
    - 当检测到按键按下时：
        - 确保存储目录存在
            - 用递增的序号保存照片
            - 在保存后短暂延时以防止重复触发
4. 异常处理：
    - 可以处理用户中断（如按Ctrl+C）
    - 捕获并显示其他可能的异常
5. 程序结束处理：
    - 停止摄像头工作
    - 关闭显示设备
    - 释放媒体缓存
    - 清理系统资源